

Arrays-Order to the Universe

The types of data structures we have been using are (

Lists-We have worked with these and they are nice because they are mutable and easy to work with. But they are not great with numbers.

Tuples-We are not spending time on them. They are like lists with a funny name but they are immutable. So they can be good if you need something not to change. But we don't use them because they don't do much with numbers.

Dictionaries-We use these a little. They are thought of key:value pairs and are created with {}. We have used these already when defining our "props" on the graphs. It lets you pass a few keyword values at once. We won't use these much but you will come across them.

Numpy arrays-we have started using these. We have seen they are easy to plot and to do math with. But they are not great with large datasets with lots of different columns and with missing data. But they are the basis for a lot of things in python so you always build off of numpy.

Pandas Dataframes-We have started these. these are like supercharged numpy arrays that give you a lot more information. If you could imagine that you could name the rows and columns in a numpy array it starts to get you there. Sort of like an excel sheet in the computer memory but more powerful. Plus they are good with dates and re-ordering. So these are good for complex datasets where we want to name variables. We will mainly be using these and building everything off of them.

But how do we think about data?

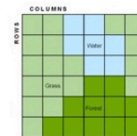
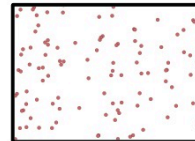
When we usually think about data we think about tabular data. This is an excel sheet. Just a table of the data we have. this is the dataframe we read in. But there are really three data types we might run into

- Tabular Data
- Vector Data
- Raster Data

The picture below explain them

Data types

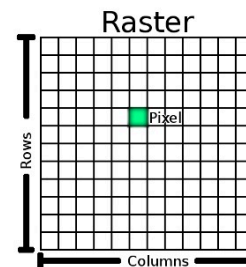
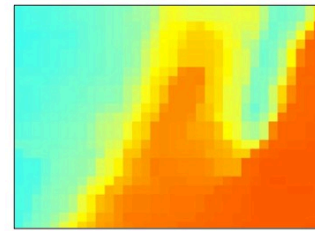
- Vector
 - Points
 - Lines
 - Polygons
- Raster
 - Digital Elevation Models (DEM)
 - Ortho imagery (aerial photography)
 - Satellite imagery
- Tabular data
 - Attributes
 - Databases



ID	Region	AREA	STATE	DATE	TOTAL POP	POP DENSITY
1	Alaska	676581.0	Alaska	1990	600,000	0.89
2	Alaska	676581.0	Alaska	2000	650,000	0.96
3	Alaska	676581.0	Alaska	2010	700,000	1.03
4	Alaska	676581.0	Alaska	2020	750,000	1.11
5	Alaska	676581.0	Alaska	2030	800,000	1.18
6	Alaska	676581.0	Alaska	2040	850,000	1.25
7	Alaska	676581.0	Alaska	2050	900,000	1.33
8	Alaska	676581.0	Alaska	2060	950,000	1.40
9	Alaska	676581.0	Alaska	2070	1,000,000	1.48
10	Alaska	676581.0	Alaska	2080	1,050,000	1.55
11	Alaska	676581.0	Alaska	2090	1,100,000	1.63
12	Alaska	676581.0	Alaska	2100	1,150,000	1.70
13	Alaska	676581.0	Alaska	2110	1,200,000	1.78
14	Alaska	676581.0	Alaska	2120	1,250,000	1.85
15	Alaska	676581.0	Alaska	2130	1,300,000	1.93

Raster data

- Areas broken into “pixels” or cells
- Each cell contains data
- Good at representing dense data:
 - Land cover
 - Elevation



Vector data are used with GIS. They are for putting lines, points, or polygons on a map. For example putting a shoreline on a map or a road or a lake. We are not going to use them much this semester.

But first today we are going to talk about Raster data and two dimensional arrays. Raster data is really a 2d array.

Today lets just work with two dimensional numpy arrays. You can have arrays of as many dimensions as you want but I have trouble comprehending at three and more dimensions.

```
In [7]: import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
import pandas as pd
np.set_printoptions(legacy='1.25') #removes the funny printing
```

```
In [8]: oneD=np.array([1,2,3,4,5,6,7,8,9,10]) # does anyone remember that band?
```

```
In [9]: oneD
```

```
Out[9]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
In [10]: #review-what will this print?
#oneD[0:10:2]
```

```
In [11]: twoD=np.array([[1,2,3,4],[5,6,7,8]])
```

```
In [12]: twoD
```

```
Out[12]: array([[1, 2, 3, 4],
               [5, 6, 7, 8]])
```

Do you see what I just did? It is 2 one dimensional arrays together to make a 2-dimensional array or table!

```
In [13]: type(twoD)
```

```
Out[13]: numpy.ndarray
```

```
In [14]: len(twoD)
```

```
Out[14]: 2
```

```
In [15]: twoD.shape
```

```
Out[15]: (2, 4)
```

```
In [16]: np.shape(twoD)
```

```
Out[16]: (2, 4)
```

```
In [17]: twoD.size
```

```
Out[17]: 8
```

You can see what you can do with the np array by typing twoD. and then tab and you see the functions available. try one

```
In [108... twoD.
```

Now lets try slicing. Remember it is rows and then columns. See if you can guess before uncommenting and running. This picture is just to help you and you don't need to load it.

```
In [4]: from IPython.display import Image  
Image(filename='array-axes.png',width=400)
```

Out[4]:

		axis 1		
		0	1	2
axis 0	0	0,0	0,1	0,2
	1	1,0	1,1	1,2
	2	2,0	2,1	2,2

```
In [18]: twoD[0,0]
```

```
Out[18]: 1
```

```
In [19]: twoD[1,0]
```

```
Out[19]: 5
```

```
In [20]: twoD[:,:]
```

```
Out[20]: array([[1, 2, 3, 4],  
               [5, 6, 7, 8]])
```

```
In [16]: #twoD[:,0]
```

```
In [111... #twoD[:,1]
```

```
In [112... #twoD[0,:]
```

```
In [113... #twoD[1,:]
```

```
In [114... #twoD[0,0]
```

```
In [115... #twoD[3,3]
```

```
In [116... #twoD[1,3]
```

np.vstack adds a row. So lets make our array bigger and keep going!

```
In [21]: twoD=np.vstack((twoD,[9,10,11,12]))
```

```
In [22]: twoD
```

```
Out [22]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12]])
```

```
In [23]: twoD=np.vstack((twoD,[13,14,15,16]))
```

```
In [24]: twoD
```

```
Out [24]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12],
                [13, 14, 15, 16]])
```

Now lets do some more slicing!

remember. For numpy it is

[start:stop:skip]

if you list multiple items and leaving one out assumes the last one is missing

so that means [1::] is one to the end by 1.

If you have a 2d array it will be [start:stop:skip,start:stop:skip] for the rows and then the columns

```
In [25]: twoD[:,:]
```

```
Out [25]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12],
                [13, 14, 15, 16]])
```

```
In [17]: #twoD[:,2,::2]
```

```
In [18]: #twoD[2:,2:]
```

```
In [19]: #twoD[1:3,1:3]
```

```
In [20]: #twoD[1::2,:]
```

Now you can set the numbers in different places.

```
In [26]: twoD[3,3]=100
```

```
In [27]: twoD
```

```
Out [27]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12],
                [13, 14, 15, 100]])
```

```
In [28]: twoD[1:3,1:3]=55
```

```
In [29]: twoD
```

```
Out [29]: array([[ 1,  2,  3,  4],
                [ 5, 55, 55,  8],
                [ 9, 55, 55, 12],
                [13, 14, 15, 100]])
```

you can use a function to set numbers!

```
In [30]: twoD[0,:]=np.arange(10,14)
```

```
In [31]: twoD
```

```
Out [31]: array([[10, 11, 12, 13],
                [ 5, 55, 55,  8],
                [ 9, 55, 55, 12],
                [13, 14, 15, 100]])
```

you can reshape the array if you want to.

```
In [32]: print (twoD.reshape(16,1))
```

```
[[ 10]
 [ 11]
 [ 12]
 [ 13]
 [  5]
 [ 55]
 [ 55]
 [  8]
 [  9]
 [ 55]
 [ 55]
 [ 12]
 [ 13]
 [ 14]
 [ 15]
 [100]]
```

```
In [33]: print (np.reshape(twoD,(1,16)))
```

```
[[ 10  11  12  13   5  55  55   8   9  55  55  12  13  14  15 100]]
```

```
In [34]: print (twoD.reshape(8,2))
```

```
[[ 10  11]
 [ 12  13]
 [  5  55]
 [ 55   8]
 [  9  55]
 [ 55  12]
 [ 13  14]
 [ 15 100]]
```

```
In [35]: print (twoD)
```

```
[[ 10  11  12  13]
 [  5  55  55   8]
 [  9  55  55  12]
 [ 13  14  15 100]]
```

The shape is back to how we had it because we never changed because we only printed it. we never set it.

Now this is where we intersect with Raster Data

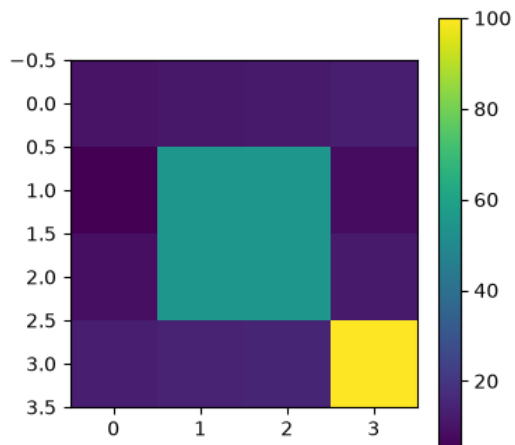
Add Color!

You can visualize the whole array! This just colors the array/grid we have by its values

This is raster data. It is like satellite data.

```
In [36]: fig,ax=plt.subplots()
fig.set_size_inches(4,4)
cax=ax.imshow(twoD)
fig.colorbar(cax)
```

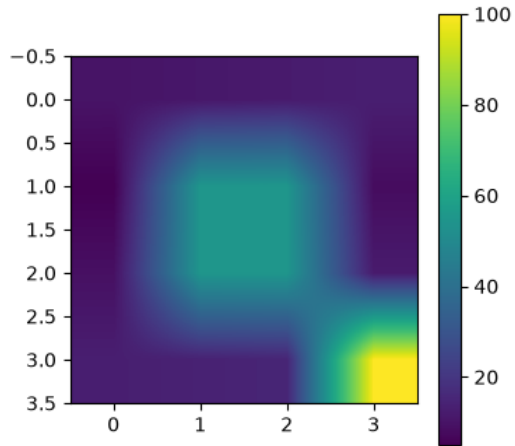
```
Out[36]: <matplotlib.colorbar.Colorbar at 0x11277e710>
```



If you want to make the edges of each box smooth we need to interpolate the data. I chose interpolation='bilinear' but there are many options https://matplotlib.org/gallery/images_contours_and_fields/interpolation_methods.html

```
In [37]: fig,ax=plt.subplots()
fig.set_size_inches(4,4)
cax=ax.imshow(twoD,interpolation='bilinear')
fig.colorbar(cax)
```

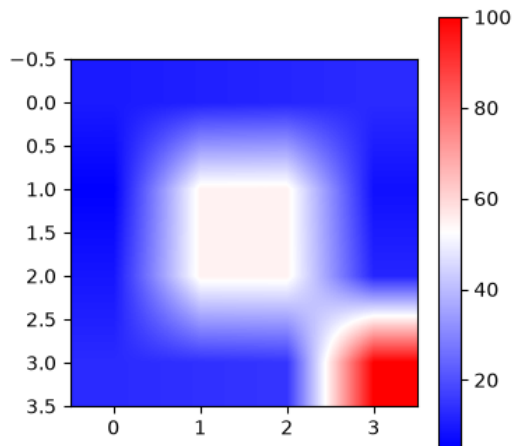
Out[37]: <matplotlib.colorbar.Colorbar at 0x1128b0e50>



We can change the colorbar. This is another keyword argument

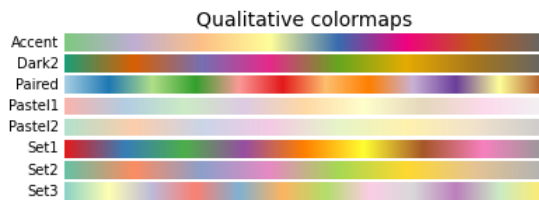
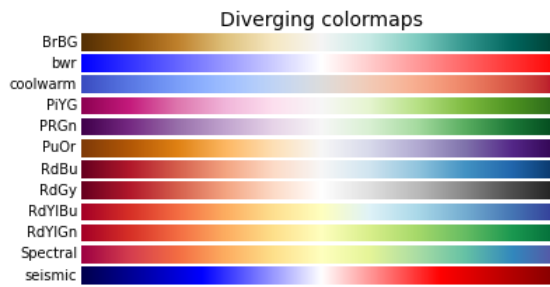
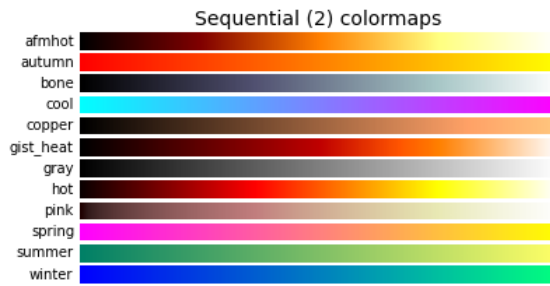
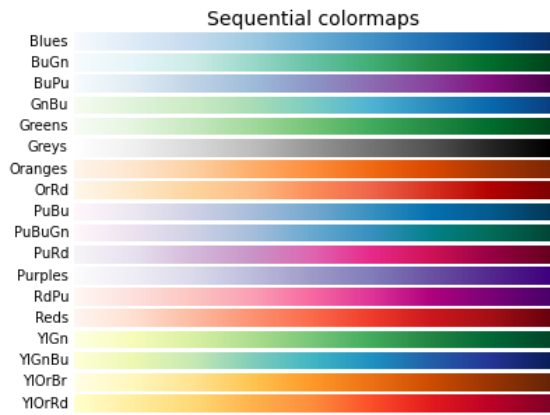
```
In [38]: fig,ax=plt.subplots()
fig.set_size_inches(4,4)
cax=ax.imshow(twoD,interpolation='bilinear',cmap='bwr')
fig.colorbar(cax)
```

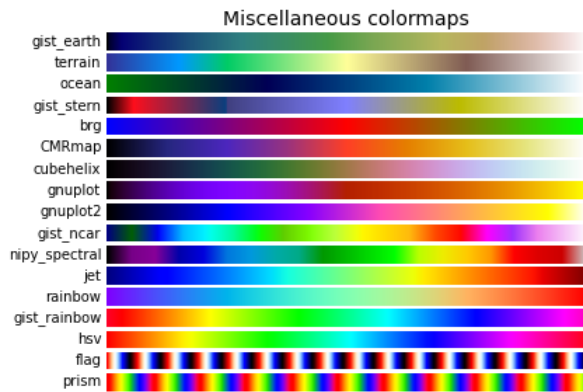
Out[38]: <matplotlib.colorbar.Colorbar at 0x112936710>



Now make your own. Here is just one [list](#). You can google colormaps python.

```
In [140... #See below for the colormap code. I also just ran the code off the web to make them ourselves.
```

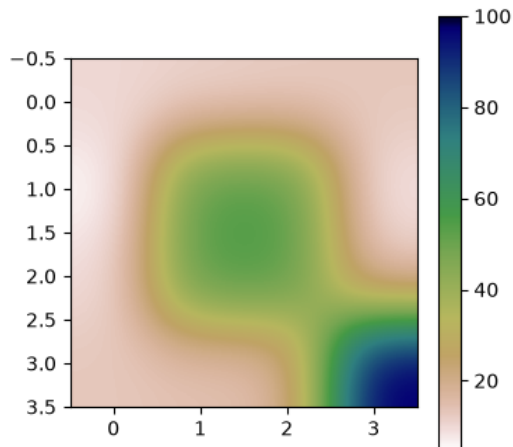




- Plot the same plot with a color bar of your choice.
- reverse the color bar by adding _r to the name.

```
In [39]: fig,ax=plt.subplots()
fig.set_size_inches(4,4)
cax=ax.imshow(twoD,cmap='gist_earth_r',interpolation='gaussian')
fig.colorbar(cax)
```

Out[39]: <matplotlib.colorbar.Colorbar at 0x112b03210>



Now we are going to intersect back with pandas and dataframes.

We are going to

- read in a csv using pandas.
- this gives us a big dataframe which we can convert to a 2 dimensional array.
- then we can plot/map it.
- This is a raster file
- The axes are telling you the array number

```
In [139]: df=pd.read_csv('my_first_csv.csv',header=None)
```

```
In [140]: twoD=df.values # this takes the dataframe and turns it into an array
```

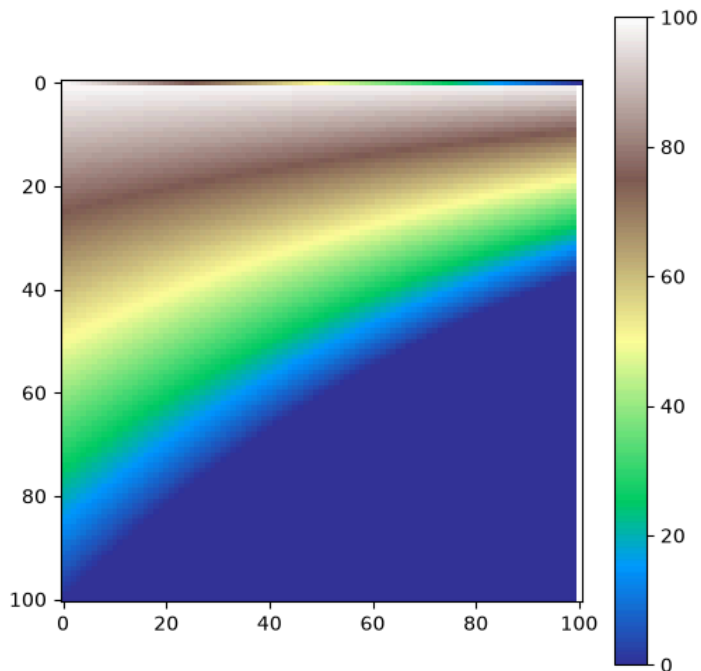
```
In [141]: print(twoD)
```

```
[[1.00000000e+02 9.90000000e+01 9.80000000e+01 ... 2.00000000e+00
 1.00000000e+00 0.00000000e+00]
[9.90000000e+01 9.89900000e+01 9.89799000e+01 ... 9.73484817e+01
 9.73219665e+01 nan]
[9.80000000e+01 9.79800000e+01 9.79598000e+01 ... 9.46969634e+01
 9.46439330e+01 nan]
...
[2.00000000e+00 1.02000000e+00 3.02000000e-02 ... 0.00000000e+00
 0.00000000e+00 nan]
[1.00000000e+00 0.00000000e+00 0.00000000e+00 ... 0.00000000e+00
 0.00000000e+00 nan]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 ... 0.00000000e+00
 0.00000000e+00 nan]]
```

```
In [143... fig,ax=plt.subplots()
fig.set_size_inches(6,6)

cax=ax.imshow(twoD,cmap='terrain',interpolation='none')
fig.colorbar(cax)
```

Out[143... <matplotlib.colorbar.Colorbar at 0x11865e710>

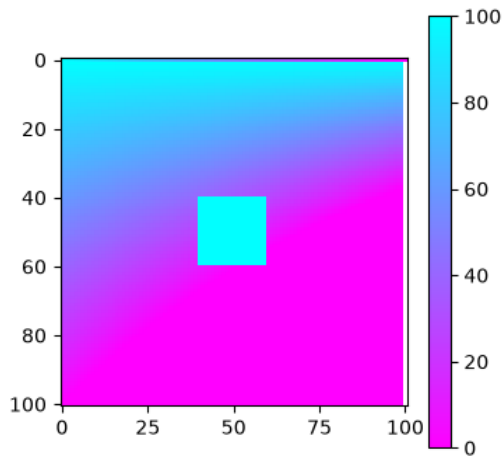


Remember we can alter the array. I am going to add a square in the middle of value 100...

```
In [144... twoD[40:60,40:60]=100
```

```
In [145... fig,ax=plt.subplots()
fig.set_size_inches(4,4)
cax=ax.imshow(twoD,cmap='cool_r',interpolation='none')
fig.colorbar(cax)
```

Out[145... <matplotlib.colorbar.Colorbar at 0x1187254d0>



In []:

Now you need to Read in Brian.csv.

It is a 2d array. Plot it with imshow and tell me what it looks like. you will be shocked at what it shows.....

<https://github.com/bmaillou/BigDataPython/blob/master/Brian.csv>

In []:

Homework hint: Think about how the data from Brian.csv is stored and how you can change it....

Now back to Tabular data.

For most of our data we usually work with tabular data. One example is if the first column is X and the remaining columns are all different Y's. Here is an example. Read in the oneX_manyY.csv file. print it to see it. Then plot all the values. This is really how we think of excel. But we give the columns nicer names

X Value	First Y Value	Second Y Value	Third Y Value	Fourth Y Value
1.0	1.0	10.0	4.0	6.0
2.0	2.0	9.0	4.0	6.0
3.0	3.0	8.0	4.0	6.0
4.0	4.0	7.0	4.0	6.0

https://github.com/bmaillou/BigDataPython/blob/master/oneX_manyY.csv

```
In [152]: df=pd.read_csv('oneX_manyY.csv')
df
```

```
Out[152]:
```

	x_values	first_y_values	second_y_values	third_y_values	fourth_y_values
0	1	1	10	4	6
1	2	2	9	4	6
2	3	3	8	4	6
3	4	4	7	4	6
4	5	5	6	4	6
5	6	6	5	4	6
6	7	7	4	4	6
7	8	8	3	4	6
8	9	9	2	4	6
9	10	10	1	4	6

Pandas is like an upgraded numpy array with column names.

This is going to make keeping track of data much nicer

using pandas we can use the column names

```
In [58]: df.columns
```

```
Out[58]: Index(['x_values', 'first_y_values', 'second_y_values', 'third_y_values',  
              'fourth_y_values'],  
              dtype='str')
```

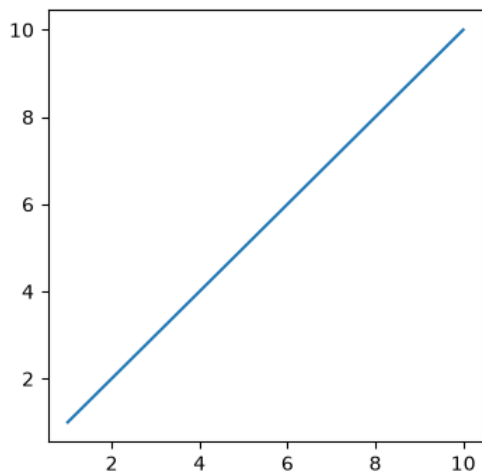
```
In [59]: df['x_values']
```

```
Out[59]: 0    1  
         1    2  
         2    3  
         3    4  
         4    5  
         5    6  
         6    7  
         7    8  
         8    9  
         9   10  
         Name: x_values, dtype: int64
```

making the plot in pandas

```
In [153]: fig,ax=plt.subplots()  
          fig.set_size_inches(4,4)  
  
          ax.plot(df['x_values'],df['first_y_values'])
```

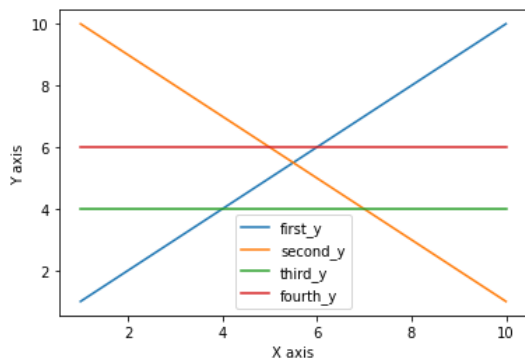
```
Out[153]: [<matplotlib.lines.Line2D at 0x117f29690>]
```



Can you add the other columns and make a legend? Using Pandas?

```
In [86]:
```

```
Out[86]: Text(0, 0.5, 'Y axis')
```



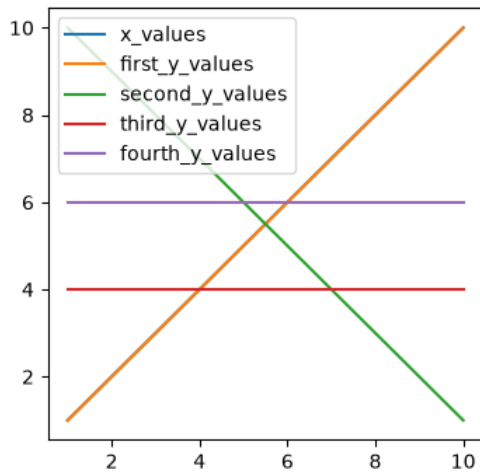
Pandas does some things to make your life easy. You can for loop over the columns. So the for loop returns the column name to col and you can pass that to ax.plot. We are going to be doing a lot more of this the next few weeks. So this is a sneak peak.

```
In [154... fig,ax=plt.subplots()
fig.set_size_inches(4,4)

for col in df:
    ax.plot(df['x_values'],df[col],label=col)

ax.legend()
```

Out[154... <matplotlib.legend.Legend at 0x119b28590>



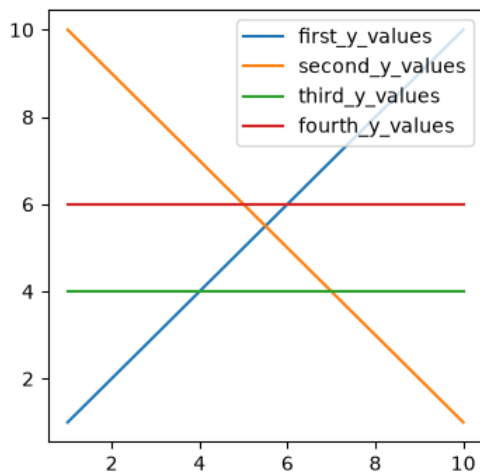
But this plots the first column versus itself. So we just need to skip the plotting the first column. We can just drop it

```
In [155... fig,ax=plt.subplots()
fig.set_size_inches(4,4)

for col in df.columns.drop('x_values'):
    ax.plot(df['x_values'],df[col],label=col)

ax.legend()
```

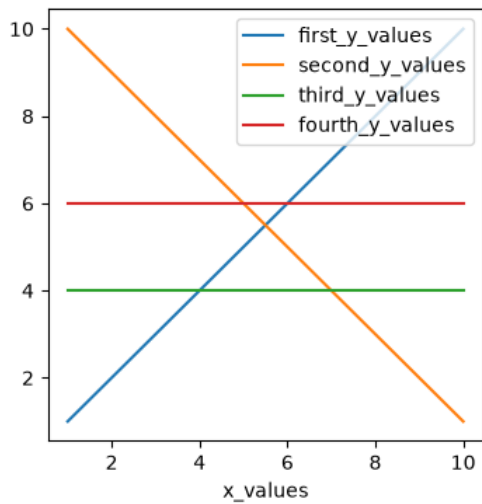
Out[155... <matplotlib.legend.Legend at 0x119bc8510>



Or we will learn this soon. You can just plot right from pandas.

```
In [156... fig,ax=plt.subplots()
fig.set_size_inches(4,4)

df.set_index('x_values').plot(ax=ax)
plt.show()
```



Mystery file

The file `mystery.csv` contains data in columns. Use what you know and plot the data! The first column is x values. The others are y values

In [143]:

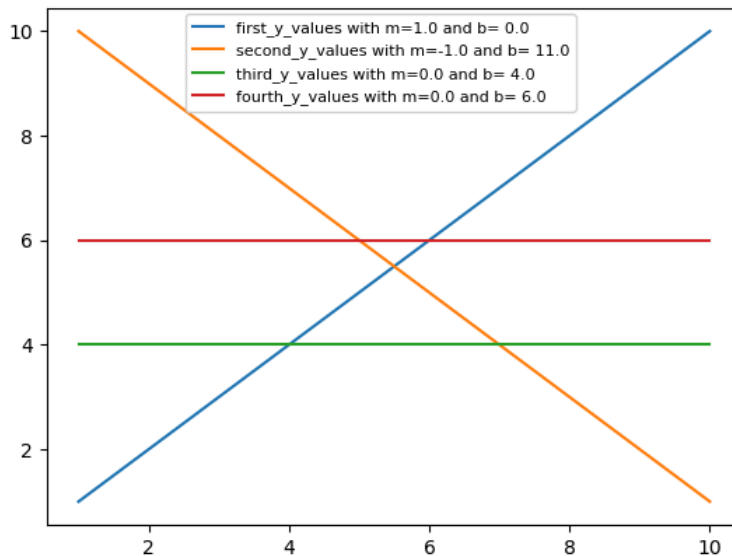
Bonus.

If you got through this quickly see if you can go back to `oneX_manyY.csv` and get the equations for each line. You could do this in a for loop and adding each equation for a line to the legend...

In [74]:

Out[74]: <matplotlib.legend.Legend at 0x13342e6d0>

Figure



Answers

My own colorbar

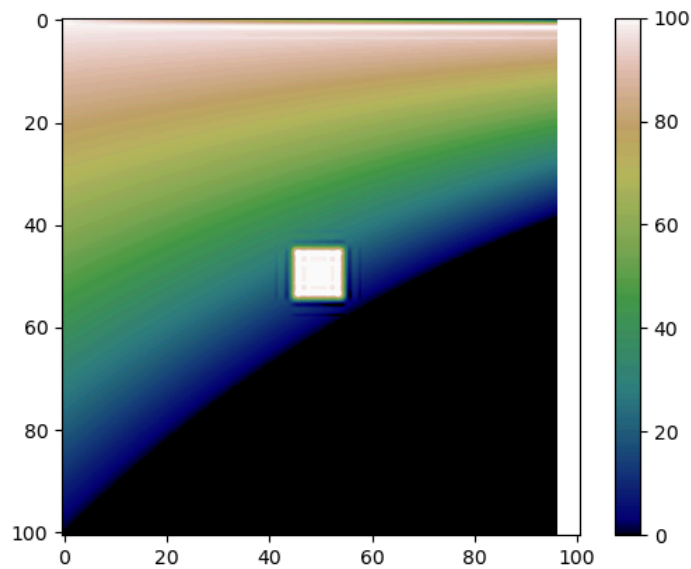
In [60]:

```
fig,ax=plt.subplots()
cax=ax.imshow(twoD,cmap='gist_earth',interpolation='bessel')
```

```
fig.colorbar(cax)
```

Out [60]: <matplotlib.colorbar.Colorbar at 0x132d5fbe0>

Figure

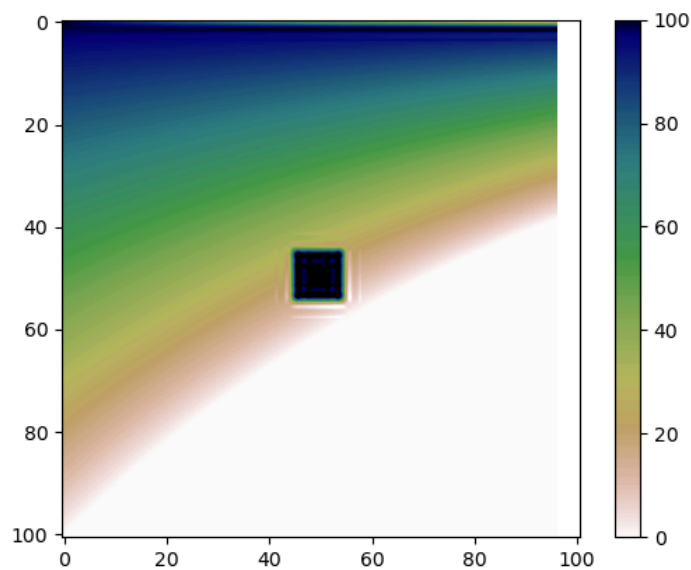


now reversed

```
In [61]: fig,ax=plt.subplots()
cax=ax.imshow(twoD,cmap='gist_earth_r',interpolation='bessel')
fig.colorbar(cax)
```

Out [61]: <matplotlib.colorbar.Colorbar at 0x132de3820>

Figure



Brian result

```
In [62]: Brian=pd.read_csv('Brian.csv',header=None)
Brian=Brian.values
```

```
fig,ax=plt.subplots()
ax.imshow(Brian,cmap='gnuplot',interpolation='none')
```

Out[62]: <matplotlib.image.AxesImage at 0x132d2e970>

Figure

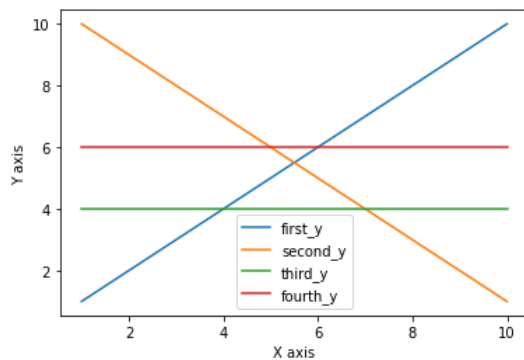


```
In [99]: fig,ax=plt.subplots()

ax.plot(df_manyY['x_values'],df_manyY['first_y_values'],label='first_y')
ax.plot(df_manyY['x_values'],df_manyY['second_y_values'],label='second_y')
ax.plot(df_manyY['x_values'],df_manyY['third_y_values'],label='third_y')
ax.plot(df_manyY['x_values'],df_manyY['fourth_y_values'],label='fourth_y')

ax.legend(loc='best')
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
```

Out[99]: Text(0, 0.5, 'Y axis')



Mystery File

```
In [53]: df_mystery=pd.read_csv('mystery.csv')
```

```
In [65]: df_mystery
```

```
Out [65]:
```

	Column_0	Column_1	Column_2
0	0.000000	0.000000e+00	1.000000
1	0.031447	3.144219e-02	0.999506
2	0.062895	6.285329e-02	0.998023
3	0.094342	9.420223e-02	0.995553
4	0.125789	1.254580e-01	0.992099
...	...	...	...
995	31.290137	-1.254580e-01	0.992099
996	31.321584	-9.420224e-02	0.995553
997	31.353032	-6.285329e-02	0.998023
998	31.384479	-3.144219e-02	0.999506
999	31.415927	4.100000e-09	1.000000

1000 rows x 3 columns

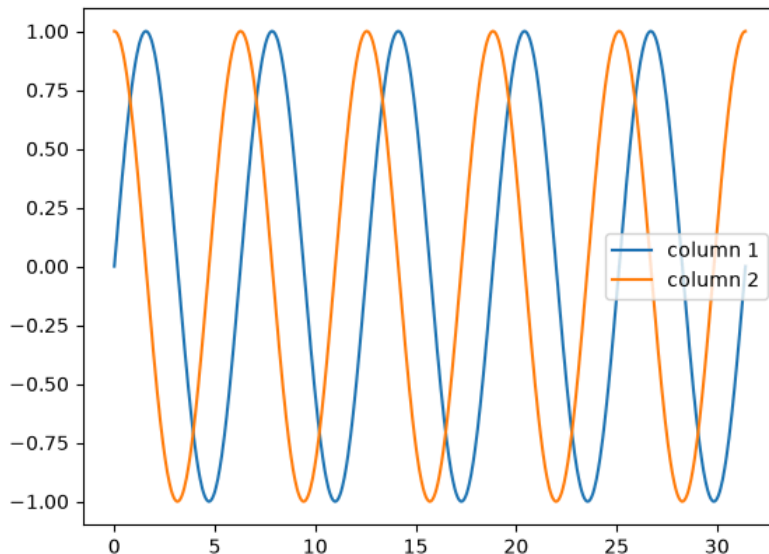
```
In [159... df_mystery=pd.read_csv('mystery.csv')
fig.set_size_inches(3,2)

fig,ax=plt.subplots()

ax.plot(df_mystery['Column_0'],df_mystery['Column_1'],label='column 1')
ax.plot(df_mystery['Column_0'],df_mystery['Column_2'],label='column 2')

ax.legend()
```

```
Out [159... <matplotlib.legend.Legend at 0x119d86190>
```



Using a for loop to get all the equations for the lines. Use linregress

```
In [160... df=pd.read_csv('oneX_manyY.csv')
fig.set_size_inches(3,3)

fig,ax=plt.subplots()

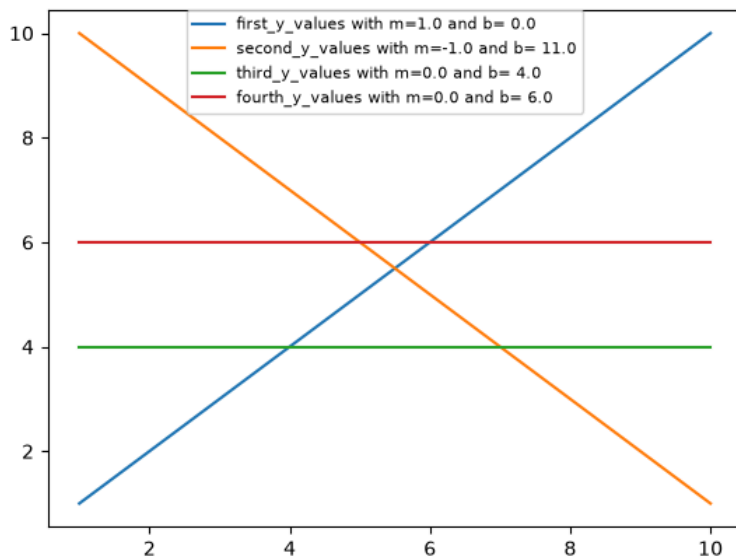
for col in df.columns.drop('x_values'):
    x=df['x_values']
    y=df[col]

    LinResult = stats.linregress(x,y)
    label='{0} with m={1} and b= {2}'.format(col,LinResult.slope,LinResult.intercept)

    ax.plot(df['x_values'],df[col],label=label)

ax.legend(loc=(.2,.8),fontsize=8)
```

Out[160... <matplotlib.legend.Legend at 0x119e2e190>



Code I copied from the web to show all the colormaps

```
In [ ]: """
Reference for colormaps included with Matplotlib.

This reference example shows all colormaps included with Matplotlib. Note that
any colormap listed here can be reversed by appending "_r" (e.g., "pink_r").
These colormaps are divided into the following categories:

Sequential:
    These colormaps are approximately monochromatic colormaps varying smoothly
    between two color tones---usually from low saturation (e.g. white) to high
    saturation (e.g. a bright blue). Sequential colormaps are ideal for
    representing most scientific data since they show a clear progression from
    low-to-high values.

Diverging:
    These colormaps have a median value (usually light in color) and vary
    smoothly to two different color tones at high and low values. Diverging
    colormaps are ideal when your data has a median value that is significant
    (e.g. 0, such that positive and negative values are represented by
    different colors of the colormap).

Qualitative:
    These colormaps vary rapidly in color. Qualitative colormaps are useful for
    choosing a set of discrete colors. For example::

        color_list = plt.cm.Set3(np.linspace(0, 1, 12))

    gives a list of RGB colors that are good for plotting a series of lines on
    a dark background.

Miscellaneous:
    Colormaps that don't fit into the categories above.
"""
import numpy as np
import matplotlib.pyplot as plt

cmaps = [('Sequential',      ['Blues', 'BuGn', 'BuPu',
                              'GnBu', 'Greens', 'Greys', 'Oranges', 'OrRd',
                              'PuBu', 'PuBuGn', 'PuRd', 'Purples', 'RdPu',
                              'Reds', 'YlGn', 'YlGnBu', 'YlOrBr', 'YlOrRd']),
          ('Sequential (2)', ['afmhot', 'autumn', 'bone', 'cool', 'copper',
                              'gist_heat', 'gray', 'hot', 'pink',
                              'spring', 'summer', 'winter']),
          ('Diverging',      ['BrBG', 'bwr', 'coolwarm', 'PiYG', 'PRGn', 'PuOr',
                              'RdBu', 'RdGy', 'RdYlBu', 'RdYlGn', 'Spectral',
                              'seismic']),
          ('Qualitative',     ['Accent', 'Dark2', 'Paired', 'Pastel1',
                              'Pastel2', 'Set1', 'Set2', 'Set3'])]
```

```

        ('Miscellaneous', ['gist_earth', 'terrain', 'ocean', 'gist_stern',
                           'brg', 'CMRmap', 'cubehelix',
                           'gnuplot', 'gnuplot2', 'gist_ncar',
                           'nipy_spectral', 'jet', 'rainbow',
                           'gist_rainbow', 'hsv', 'flag', 'prism']])

nrows = max(len(cmap_list) for cmap_category, cmap_list in cmaps)
gradient = np.linspace(0, 1, 256)
gradient = np.vstack((gradient, gradient))

def plot_color_gradients(cmap_category, cmap_list):
    fig, axes = plt.subplots(nrows=nrows)
    fig.subplots_adjust(top=0.95, bottom=0.01, left=0.2, right=0.99)
    axes[0].set_title(cmap_category + ' colormaps', fontsize=14)

    for ax, name in zip(axes, cmap_list):
        ax.imshow(gradient, aspect='auto', cmap=plt.get_cmap(name))
        pos = list(ax.get_position().bounds)
        x_text = pos[0] - 0.01
        y_text = pos[1] + pos[3]/2.
        fig.text(x_text, y_text, name, va='center', ha='right', fontsize=10)

    # Turn off *all* ticks & spines, not just the ones with colormaps.
    for ax in axes:
        ax.set_axis_off()

for cmap_category, cmap_list in cmaps:
    plot_color_gradients(cmap_category, cmap_list)

plt.show()

```

In []: